12. Keylime 보안 및 오류 처리 메커니즘 분석

12.1 개요

Keylime은 TPM 기반 원격 증명 시스템으로서 높은 수준의 보안성과 안정성을 요구합니다. 이 문서는 Keylime의 보안 메커니즘과 오류 처리 시스템을 상세히 분석하여, 시스템의 신뢰성과 복구 능력을 이해할 수 있도록 합니다.

12.2 실패 관리 시스템 (keylime/failure.py)

12.2.1 핵심 클래스 구조

Keylime의 실패 관리 시스템은 세 가지 핵심 클래스로 구성됩니다:

SeverityLabel 클래스

```
@functools.total_ordering

class SeverityLabel:

name: str # 심각도 레이블 이름 (예: "critical", "warning")

severity: int # 심각도 수준 (숫자가 높을수록 심각)
```

기능:

- 이벤트의 심각도를 동적으로 할당
- 설정 기반 심각도 정책 지원
- 비교 연산자를 통한 심각도 순서 정렬

Component 열거형

```
class Component(enum.Enum):
QUOTE_VALIDATION = "qoute_validation"  # TPM Quote 검증
PCR_VALIDATION = "pcr_validation"  # PCR 값 검증
MEASURED_BOOT = "measured_boot"  # 측정 부팅 검증
IMA = "ima"  # IMA 무결성 검증
INTERNAL = "internal"  # 내부 시스템 오류
TEE = "tee_attestation"  # TEE 증명
DEFAULT = "default"  # 기본 컴포넌트
```

역할:

- Keylime의 주요 검증 컴포넌트 식별
- 실패 이벤트의 발생 위치 추적
- 컴포넌트별 오류 분류 및 처리

Event 클래스

```
class Event:
event_id: str # 계층적 이벤트 ID (예: "ima.hash_mismatch")
severity_label: SeverityLabel # 심각도 레이블
context: str # JSON 형태의 컨텍스트 정보
recoverable: bool # 복구 가능 여부
```

이벤트 ID 생성 규칙:

```
component.[sub_component].event_id

예시: "ima.file_signature.verification_failed"
```

12.2.2 Failure 클래스 - 통합 실패 관리

```
class Failure:

events: List[Event] # 발생한 모든 이벤트

recoverable: bool # 전체 실패의 복구 가능성

highest_severity: Optional[SeverityLabel] # 최고 심각도

highest_severity_event: Optional[Event] # 최고 심각도 이벤트
```

핵심 메서드:

1. **add_event()** - 이벤트 추가

```
def  add_event(self, event_id: str, context: Union[str, Dict],

recoverable: bool, sub_components: Optional[List[str]] = None):
```

2. **merge()** - 다른 Failure 객체와 병합

```
def  merge(self, other: "Failure") -> None:
```

12.2.3 심각도 정책 시스템

기본 심각도 레이블 (낮음 → 높음)

```
["info", "notice", "warning", "error", "critical", "alert", "emergency"]
```

정책 설정 예시

```
severity_policy = [
{"event_id": "ima\\..*", "severity_label": "critical"},
{"event_id": "pcr_validation\\..*", "severity_label": "error"},
{"event_id": "internal\\..*", "severity_label": "warning"}
]
```

동적 심각도 할당:

- 정규표현식 기반 이벤트 ID 매칭
- 설정 파일을 통한 정책 변경 가능
- 복구 불가능한 이벤트는 자동으로 최고 심각도 할당

12.3 철회 알림 시스템 (keylime/revocation_notifier.py)

12.3.1 알림 방식

Keylime은 세 가지 철회 알림 방식을 지원합니다:

1. **ZeroMQ** - 메시지 큐 기반 알림
2. **Webhook** - HTTP 기반 알림
3. **Agent** - 직접 에이전트 알림

12.3.2 WebhookNotificationManager 클래스

```
class WebhookNotificationManager:
_shutdown_event: threading.Event # 종료 신호
_workers: Set[threading.Thread] # 활성 워커 스레드
_workers_lock: threading.Lock # 스레드 동기화
```

핵심 기능:

1. 비동기 웹훅 알림

```
def  notify_webhook(self, tosend: Dict[str, Any]) -> None:
```

특징:

- 별도 워커 스레드에서 실행
- 재시도 메커니즘 (지수 백오프 지원)
- TLS 상호 인증 지원
- 종료 시 우아한 처리

2. 재시도 전략

```
max_retries = config.getint("verifier", "max_retries")
interval = config.getfloat("verifier", "retry_interval")
exponential_backoff = config.getboolean("verifier", "exponential_backoff")
```

종료 모드 최적화:

- 종료 시 재시도 횟수 제한 (최대 3회)
- 재시도 간격 단축 (최대 2초)

- 중요한 철회 알림의 전송 보장

3. 우아한 종료 (Graceful Shutdown)

```
def shutdown_workers(self) -> None:
```

프로세스:

1. 모든 워커에게 종료 신호 전송
2. 30초 타임아웃으로 워커 완료 대기
3. 중요한 보안 알림 전송 보장
4. 워커 정리 및 리소스 해제

12.3.3 ZeroMQ 브로커 시스템

브로커 프로세스 관리

```
def start_broker() -> None:
def stop_broker() -> None:
```

아키텍처:

```
[Verifier] → [IPC Socket] → [ZeroMQ Proxy] → [TCP Socket] → [Subscribers]
```

보안 특징:

- IPC 소켓 권한 제어 (0o700)
- 별도 프로세스 그룹으로 격리
- 신호 처리를 통한 안전한 종료

12.3.4 메시지 서명 검증

```
def process_revocation(revocation: Dict[str, Any],
callback: Callable, cert_path: str) -> None:
```

검증 과정:

1. 철회 인증서 로드 (지연 로딩)
2. RSA 서명 검증
3. 메시지 무결성 확인
4. 검증된 메시지만 콜백 실행

보안 검사:

```
if not crypto.rsa_verify(cert_key, revocation["msg"].encode("utf-8"),
revocation["signature"].encode("utf-8")):
logger.error("Invalid revocation message signature")
```

12.4 로깅 및 모니터링 시스템 (keylime/keylime_logging.py)

12.4.1 구조화된 로깅

로거 초기화

```
def init_logging(loggername: str) -> Logger:
logger = logging.getLogger(f"keylime.{loggername}")
```

특징:

- 계층적 로거 네임스페이스 (`keylime.component`)
- 외부 라이브러리 로그 레벨 제어
- Tornado 로그 비활성화 (상세 로그는 `keylime.web` 에서 처리)

요청 ID 추적

```
request_id_var: contextvars.ContextVar[str] = contextvars.ContextVar("request_id
```

```
class RequestIDFilter(logging.Filter):
def filter(self, record: "LogRecord") -> bool:
reqid = request_id_var.get("")
record.reqid = reqid
record.reqidf = f"(reqid={reqid})" if reqid else ""
```

기능:

- 비동기 요청 간 컨텍스트 유지
- 분산 추적을 위한 요청 ID 자동 삽입
- 로그 메시지에 요청 ID 자동 포함

12.4.2 HTTP 응답 로깅

```
def log_http_response(logger: Logger, loglevel: int,
response_body: Dict[str, Any]) -> bool:
```

표준화된 응답 로깅:

- 응답 코드와 상태 메시지 추출
- 예상 형식 검증 (`results` , `code` , `status` 필드)
- 오류 응답의 구조화된 로깅

12.4.3 보안 로깅 고려사항

민감 정보 보호:

- 암호화 키나 인증서는 로그에 기록하지 않음
- 디버그 모드에서만 상세 정보 출력
- 요청/응답 페이로드의 선택적 로깅

12.5 암호화 및 보안 메커니즘 (keylime/crypto.py)

12.5.1 RSA 키 관리

키 가져오기 및 검증

```
def rsa_import_pubkey(pubkey: Union[str, bytes]) -> RSAPublicKey:
def rsa_import_privkey(privkey: Union[str, bytes],
password: Optional[bytes] = None) -> RSAPrivateKey:
```

보안 특징:

- 타입 안전성 검증
- RSA 키 타입 강제 확인
- 패스워드 보호 개인키 지원

X.509 인증서 처리

```
def x509_import_pubkey(pubkey: bytes) -> RSAPublicKey:
```

12.5.2 디지털 서명

RSA 서명 생성

```
def rsa_sign(key: RSAPrivateKey, message: bytes,
paddingt: str = "internal") -> bytes:
```

패딩 방식:

- `internal` : PSS 패딩 (권장, 더 안전)
- `default` : PKCS1v15 패딩 (호환성)

서명 검증

```
def rsa_verify(public_key: RSAPublicKey, message: bytes,
signature: bytes) -> bool:
```

검증 과정:

1. Base64 디코딩
2. PSS 패딩으로 서명 검증
3. SHA-256 해시 알고리즘 사용
4. 예외 처리를 통한 안전한 실패

12.6 설정 기반 보안 정책 (keylime/config.py)

12.6.1 환경 변수 보안

불린 값 검증

```
def  environ_bool(env_name: str, default: bool) -> bool:
```

안전한 환경 변수 처리:

- 허용된 값만 수용 (`on/true/1` , `off/false/0`)
- 잘못된 값에 대한 명시적 예외 발생
- 기본값 지원

테스트 모드 보안

```
TEST_MODE = environ_bool("KEYLIME_TEST", False)
```

테스트 모드 제한사항:

- EK 인증서 검증 비활성화
- 보안 마운트 생성 안함
- 작업 디렉토리 변경

12.6.2 설정 파일 계층

설정 파일 우선순위

```
CONFIG_FILES = {
"verifier": ["/etc/keylime/verifier.conf", "/usr/etc/keylime/verifier.conf"],
```

```
"tenant": ["/etc/keylime/tenant.conf", "/usr/etc/keylime/tenant.conf"],
# ...
}
```

설정 조각 (Configuration Snippets)

```
CONFIG_SNIPPETS_DIRS = {
"verifier": ["/usr/etc/keylime/verifier.conf.d", "/etc/keylime/verifier.conf.d"]
# ...
}
```

보안 이점:

- 세분화된 설정 관리
- 패키지 업데이트 시 사용자 설정 보존
- 컴포넌트별 격리된 설정

12.7 오류 처리 패턴

12.7.1 사용자 정의 예외 (keylime/tenant.py)

```
class UserError(Exception):
"""스택 트레이스를 숨기는 특별한 예외"""
pass
```

사용 사례:

- 사용자 입력 오류
- 설정 문제
- 네트워크 연결 실패

12.7.2 TLS 연결 오류 처리

SSL 인증서 검증 오류

```
except requests.exceptions.SSLError as ssl_error:
if  "TLSV1_ALERT_UNKNOWN_CA" in str(ssl_error):
raise UserError(
"Keylime agent does not recognize mTLS certificate from tenant. "
"Check if agents trusted_client_ca is configured correctly"
) from ssl_error
```

연결 오류 분류 (keylime/tornado_requests.py)

```
except httpclient.HTTPError as e:
return TornadoResponse(e.response.code, e.response.body)
except ConnectionError as e:
return TornadoResponse(599, f"Connection error: {str(e)}")
except ssl.SSLError as e:
return TornadoResponse(599, f"SSL connection error: {str(e)}")
```

12.7.3 복구 메커니즘

재시도 알고리즘

- 지수 백오프 지원
- 최대 재시도 횟수 제한
- 설정 가능한 재시도 간격

상태 복구

- 에이전트 상태 재동기화
- TPM 상태 복원
- 인증서 갱신

12.8 보안 취약점 대응

12.8.1 입력 검증

설정 값 검증

```
if value != "default":
raise Exception(f"To use tls_dir=generate, the following options must be set to
```

파일 경로 검증

```
if not os.path.exists(tls_dir):
raise Exception(f"It appears that the specified tls_dir does not exist: {tls_dir
```

12.8.2 암호화 강도 검증

키 크기 검증

```
except ssl.SSLError as exc:
if exc.reason == "EE_KEY_TOO_SMALL" and logger:
logger.error(
"Higher key strength is required for keylime "
"running on this system."
)
```

12.8.3 권한 및 접근 제어

디렉토리 권한 검증

```
if os.stat(dir_name).st_mode & 0o777 != 0o700:
msg = f"{dir_name} present with wrong permissions"
logger.error(msg)
raise Exception(msg)
```

12.9 모니터링 및 감사

12.9.1 보안 이벤트 로깅

중요 보안 이벤트:

- 인증 실패

- 인증서 검증 실패
- TPM 상태 변경
- 철회 이벤트 발생

12.9.2 성능 모니터링

메트릭 수집:

- 응답 시간
- 오류율
- 재시도 횟수
- 연결 상태

12.9.3 감사 추적

추적 정보:

- 요청 ID 기반 추적
- 컴포넌트 간 호출 체인
- 사용자 작업 기록
- 시스템 상태 변경

12.10 결론

Keylime의 보안 및 오류 처리 메커니즘은 다음과 같은 특징을 가집니다:

강점:

1. **계층적 실패 관리** - 컴포넌트별 세분화된 오류 분류
2. **동적 심각도 정책** - 설정 기반 유연한 심각도 할당

3. **강력한 철회 시스템** - 다중 채널 알림 및 재시도 메커니즘

4. **구조화된 로깅** - 요청 추적 및 보안 이벤트 기록

5. **암호화 보안** - 강력한 RSA 서명 및 검증

6. **우아한 종료** - 중요 알림의 안전한 전송 보장

보안 고려사항:

1. **입력 검증** - 모든 외부 입력에 대한 엄격한 검증

2. **권한 제어** - 파일 시스템 권한 및 접근 제어

3. **암호화 강도** - 충분한 키 크기 및 안전한 알고리즘

4. **감사 추적** - 모든 보안 관련 활동의 로깅

이러한 메커니즘들이 통합되어 Keylime의 높은 보안성과 안정성을 보장합니다.